

IMPLEMENTATION STRATEGY For KCA Connect Agentic AI

BSC INFORMATION TECHNOLOGY
IGNATIUS LUMOSI
22/05647
GRIFFIN KENGA

Table of Contents

1. Introduction	1
2. Implementation Objectives.....	2
3. Implementation Approach	3
4. System Organization	4
4.1. Software Components	4
4.2. Hardware Requirements	5
4.3. System Architecture Diagram	5
5. Description of Implementation	7
5.1. Changeover Strategy.....	7
5.2. Installation Procedure	7
5.3. Local Development Setup	8
6. Major Implementation Tasks.....	9
7. Implementation Support	11
7.1. Hardware	11
7.2. Software.....	11
7.3. Facilities	12
Appendices	12

Abstract

This chapter forms part of the final year project report for KCA Connect Agentic AI, an intelligent conversational assistant developed for KCA University. The system leverages Retrieval-Augmented Generation (RAG), large language models (LLMs), and a modern web stack to enable students, faculty, and staff to access institutional information through natural language queries.

1. Introduction

KCA Connect Agentic AI is an intelligent conversational assistant developed to help KCA University students, faculty, and staff access institutional information through natural language queries. The system leverages Retrieval-Augmented Generation (RAG) to deliver accurate, context-aware responses drawn from official university documents, including timetables, fee structures, exam schedules, and general campus information.

This chapter describes the strategy used to move the system from development to deployment. It outlines the approach taken to install, configure, and stabilise all system components while minimising disruption to existing university operations.

2. Implementation Objectives

The primary objectives of the implementation phase are:

1. To deploy a fully functional AI-powered chat interface accessible to KCA University users.
2. To ingest and index all relevant institutional documents into the Qdrant vector database.
3. To integrate third-party services (Google Gemini LLM, Groq, Cerebras, Supabase, Qdrant Cloud) in a secure and maintainable configuration.
4. To onboard users through clear documentation and guided sign-up flows.
5. To ensure system stability through health checks, monitoring, and fallback mechanisms.

3. Implementation Approach

A phased changeover strategy was adopted for implementing KCA Connect AI. This approach allows the system to be introduced gradually, validating functionality at each stage before full rollout, thereby reducing the risk of disruption.

The phases are as follows:

Phase	Description
Phase 1 – Infrastructure Setup	Configure cloud and local services (Supabase, Qdrant Cloud, environment variables).
Phase 2 – Backend Deployment	Deploy the FastAPI backend with RAG pipeline on a server or local environment.
Phase 3 – Frontend Deployment	Build and serve the React application, connecting it to the backend API.
Phase 4 – Data Ingestion	Ingest official KCA University documents into the vector knowledge base.
Phase 5 – User Acceptance Testing	Conduct testing with representative student and staff users.
Phase 6 – Go-Live	Open system access to all intended users at KCA University.

4. System Organization

The KCA Connect AI system is composed of four primary layers that work in concert to deliver intelligent responses to users.

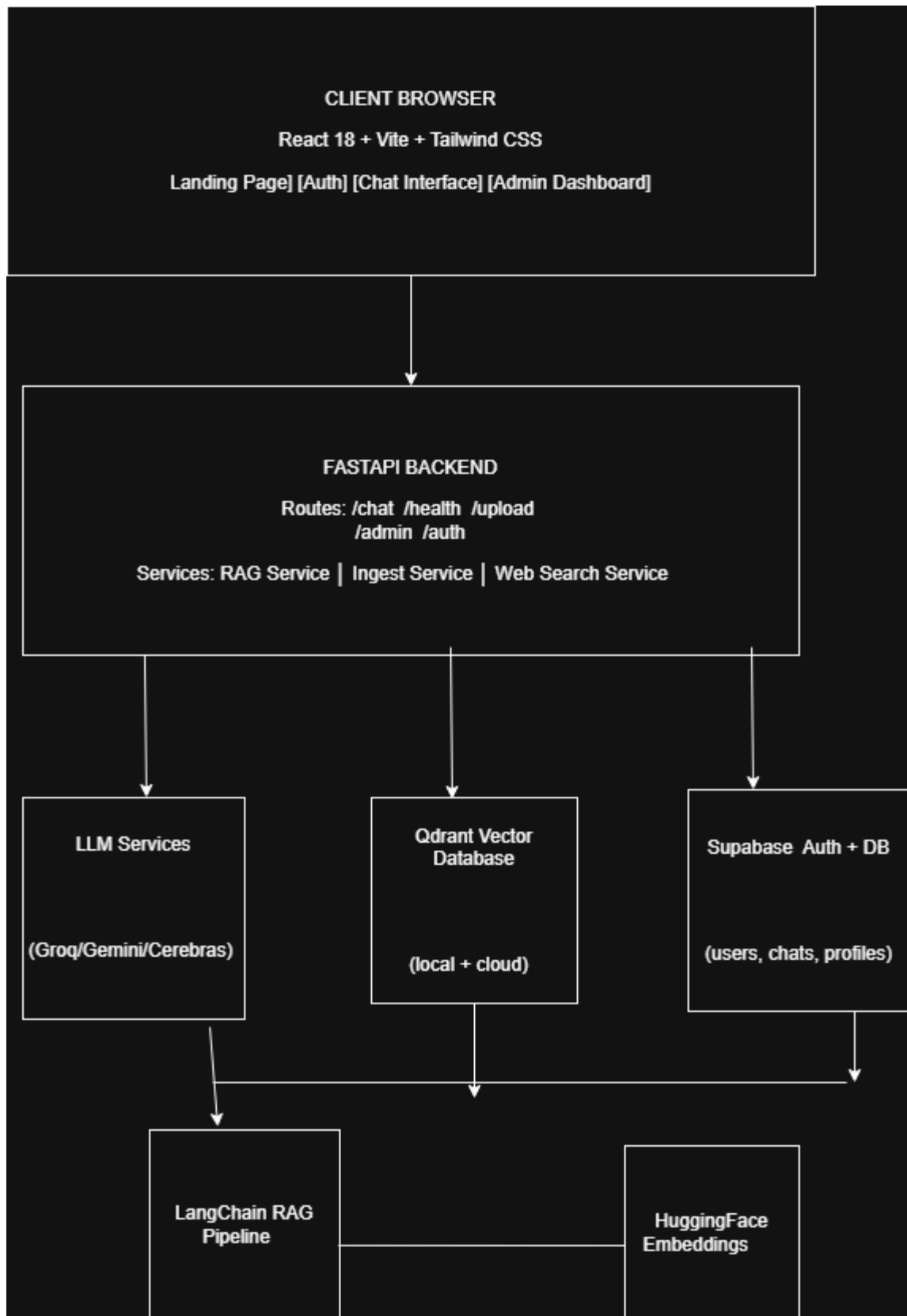
4.1. Software Components

Component	Technology	Role
Frontend	React 18, Vite, Tailwind CSS	User interface — chat, authentication, admin dashboard
Backend API	FastAPI (Python)	REST API, RAG orchestration, business logic
RAG Pipeline	LangChain	Query embedding, retrieval, LLM generation
Vector Database	Qdrant (local + cloud)	Stores and retrieves document embeddings
LLM Services	Google Gemini 1.5 Flash, Groq, Cerebras	Natural language generation with fallback routing
Auth & Database	Supabase (PostgreSQL)	User authentication, profile storage, chat history
Embeddings Model	HuggingFace `all-MiniLM-L6-v2`	Converts text to semantic vectors

4.2. Hardware Requirements

Component	Specification
Server / Host Machine	Minimum 4-core CPU, 8 GB RAM, 20 GB storage
Client Devices	Any device with a modern web browser (Chrome, Firefox, Edge, Safari)
Network	Stable internet connection (for cloud services)

4.3. System Architecture Diagram



5. Description of Implementation

5.1. Changeover Strategy

A phased parallel changeover was used. During the development and testing phases, the system operated alongside conventional manual methods of obtaining university information (e.g., visiting the university portal, emailing departments). This ensured no interruption to users while the new system was being validated.

Once the system passed all acceptance tests, the phased go-live was initiated — beginning with a limited user group (beta testers), then expanding to the general KCA student and staff population.

5.2. Installation Procedure

Step 1 — Clone the Repository

```
git clone https://pinchez420@github.com  
cd "final year proj"
```

Step 2 — Configure Environment Variables

```
cd backend  
cp .env.example .env  
# Edit .env to include:  
# GOOGLE_API_KEY, GROQ_API_KEY, CEREBRAS_API_KEY  
# SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_KEY  
# QDRANT_URL, QDRANT_CLOUD_URL, QDRANT_CLOUD_API_KEY  
# COLLECTION_NAME, EMBEDDING_MODEL, DEFAULT_LLM
```

Step 3 — Start Services (Docker)

```
docker-compose up --build
```

This starts:

- Frontend at `http://localhost:5173`

- Backend API at `http://localhost:8000`
- Qdrant at `http://localhost:6333`

Step 4 — Ingest Knowledge Base Documents

```
cd backend  
  
python scripts/ingest.py
```

Step 5 — Verify Deployment

- Visit `http://localhost:8000/health` to confirm all services report healthy status.
- Visit `http://localhost:5173` to confirm the UI loads correctly.

5.3. Local Development Setup

Frontend:

```
cd frontend  
  
npm install  
  
npm run dev
```

Backend:

```
cd backend  
  
python -m venv .venv  
  
source .venv/bin/activate  
  
pip install -r requirements.txt  
  
uvicorn main:app --reload
```

6. Major Implementation Tasks

The table below describes the major tasks involved in implementing the KCA Connect AI system.

#	Task	Personnel	Resources Required	Success Criteria
1	Planning & Coordination	Project developer, supervisor	Project plan, timeline	Approved implementation schedule
2	Infrastructure Setup	Developer	Cloud accounts (Supabase, Qdrant Cloud, Google AI Studio, Groq, Cerebras), environment config	All services accessible and authenticated
3	Backend Development & Deployment	Developer	Python 3.11+, FastAPI, LangChain, Qdrant client, server or local machine	`/health` endpoint returns all services healthy
4	Frontend Development & Deployment	Developer	Node.js 18+, Vite, React 18, Tailwind CSS	UI accessible and communicates with backend

5	Document Ingestion	Developer	Official KCA University PDF/TXT documents, HuggingFace model	Qdrant collection populated; semantic search returns relevant results
6	User Training & Onboarding	Developer	User manual, walkthrough sessions	Users can independently register, sign in, and use chat
7	User Acceptance Testing (UAT)	Developer + beta users	Test cases, feedback forms	System passes all test cases; users rate experience positively
8	Performance Optimisation	Developer	Profiling tools, monitoring	Response time under 5 seconds for average queries
9	Go-Live & Monitoring	Developer	Hosting infrastructure, logging	System stable for 48 hours post-launch with no critical errors

7. Implementation Support

7.1. Hardware

Item	Purpose
Development laptop / workstation	Building, testing, and running local instances of all services
Cloud server (optional)	Hosting for production deployment

7.2. Software

Software / Service	Purpose
Python 3.11+	Backend runtime environment
Node.js 18+	Frontend build and dev server
Docker & Docker Compose	Service orchestration and containerisation
Supabase	PostgreSQL database, user authentication
Qdrant Cloud	Cloud vector database for production
Google AI Studio (Gemini API)	Primary LLM provider
Groq API	Fallback LLM (fast inference)
Cerebras API	Secondary LLM fallback
HuggingFace (`all-MiniLM-L6-v2`)	Sentence embedding model
LangChain	RAG pipeline framework
Git	Version control
VS Code / Cursor	Code editor and IDE

7.3. Facilities

Facility	Purpose
Development workstation	Local development and testing environment
Internet connection	Access to cloud APIs and Qdrant Cloud
Browser (Chrome recommended)	Testing the frontend application

Appendices

Appendix A: Screenshots of the Running System

Screenshots of the deployed system are included in the accompanying System User Manual (Chapter 8).

Appendix B: Sample Code — RAG Query Flow

```
# backend/app/services/rag_service.py (excerpt)

async def query(self, user_message: str, user_context: dict = None) -> str:

    # 1. Embed the query

    query_embedding = self.embeddings.embed_query(user_message)

    # 2. Retrieve relevant documents from Qdrant

    results = self.qdrant_client.search(

        collection_name=self.collection_name,

        query_vector=query_embedding,

        limit=5

    )
```

```

# 3. Build context from retrieved chunks

context = "\n\n".join([r.payload["text"] for r in results])

# 4. Generate response with LLM

response = await self.llm.ainvoke(

    self.prompt_template.format(context=context, question=user_message)

)

return response.content

```

Appendix C: Environment Variables Reference

Variable	Description
`QDRANT_URL`	Local Qdrant instance URL
`QDRANT_CLOUD_URL`	Qdrant Cloud endpoint
`QDRANT_CLOUD_API_KEY`	API key for Qdrant Cloud
`COLLECTION_NAME`	Name of the vector collection
`EMBEDDING_MODEL`	HuggingFace embedding model name
`GOOGLE_API_KEY`	Google Gemini API key
`CEREBRAS_API_KEY`	Cerebras LLM API key
`GROQ_API_KEY`	Groq LLM API key
`DEFAULT_LLM`	Default LLM provider (`groq`, `gemini`, `cerebras`)
`SUPABASE_URL`	Supabase project URL
`SUPABASE_ANON_KEY`	Supabase anonymous public key
`SUPABASE_SERVICE_KEY`	Supabase service role key (admin)

References

6. FastAPI Documentation. (2024). FastAPI - Modern, fast, web framework for Python.
<https://fastapi.tiangolo.com/>
7. LangChain Documentation. (2024). LangChain Python Reference.
<https://python.langchain.com/>
8. Qdrant Documentation. (2024). Qdrant Vector Database. <https://qdrant.tech/documentation/>
9. Google AI Studio. (2024). Gemini API documentation. <https://ai.google.dev/>
10. Supabase Documentation. (2024). Supabase - Open Source Firebase Alternative.
<https://supabase.com/docs>
11. HuggingFace. (2024). Sentence Transformers: all-MiniLM-L6-v2.
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
12. React Documentation. (2024). React 18 — A JavaScript library for building user interfaces.
<https://react.dev/>